Laboratório 3 — Observabilidade e Resposta a Incidentes (Prometheus + Grafana)

Apostila de apoio — Análise e Desenvolvimento de Sistemas

1. Visão geral do laboratório

1.1 O que você vai fazer

Neste laboratório você vai:

1. Entender os três pilares da **observabilidade** (métricas, logs, traces) e por que este laboratório foca em métricas.
2. Adicionar, ao `main.tf` que você já construiu nos Labs 1 e 2, três novos serviços: **cAdvisor** (coleta métricas dos containers), **Prometheus** (armazena essas métricas) e **Grafana** (exibe um dashboard).
3. Conectar o Prometheus como fonte de dados no Grafana e montar um painel de acompanhamento de CPU.
4. Simular artificialmente um pico de uso de CPU e observar o dashboard reagir em tempo real.
5. Documentar esse incidente simulado em um **postmortem sem culpa** (*blameless postmortem*), no formato usado por equipes DevOps reais.

> **Cenário:** como Administrador(a) de Ambientes da StackUp Ltda., agora você precisa dar visibilidade ao ambiente que já provisionou e automatizou — sem observabilidade, ninguém saberia se esse "ambiente de testes" quebrou de madrugada. Veja o documento "Instruções Gerais" para o cenário completo e as regras de uso de IA como ferramenta de trabalho.

> **Trabalho em dupla:** última etapa com a mesma dupla e o mesmo repositório dos Labs 1 e 2. Troque de papel mais uma vez — quem ainda não editou o `main.tf` no VS Code nos Labs anteriores assume agora a criação da rede, do cAdvisor, do Prometheus e do Grafana, enquanto o outro acompanha as explicações e confere cada `terraform apply`. O exercício assíncrono do alerta (seção 7, item 4) e o entregável final (seção 9) — mini-postmortem, print do alerta, respostas de exercícios — continuam com contribuição individual de cada integrante, mesmo que o repositório e a infraestrutura sejam da dupla.

1.2 Objetivo de aprendizagem

Ao final deste laboratório, você deve ser capaz de explicar e demonstrar:

- Por que não basta ter uma infraestrutura provisionada (Lab 1) e uma pipeline que a mantém atualizada (Lab 2) — é preciso também saber, em tempo real, se ela está saudável.
- Como métricas fluem de um container até um dashboard: `cAdvisor → Prometheus → Grafana`.
- A diferença entre **detectar** um problema (via dashboard/alerta) e **responder** a ele (diagnóstico, mitigação, documentação).
- O papel de um postmortem sem culpa na melhoria contínua de um sistema em produção.

1.3 Onde este laboratório se encaixa

```
[1] Provisionar        →   [2] Automatizar        →   [3] Observar e reagir
    a infraestrutura           o deploy                   a incidentes
    (Terraform/Docker)         (CI/CD)                    (Prometheus/Grafana)
                                                           ▲ você está aqui
```

No Lab 1 você criou a infraestrutura; no Lab 2, ela passou a se atualizar sozinha a cada mudança. Mas nenhum dos dois laboratórios até aqui respondia a uma pergunta essencial: **como saber se algo deu errado?** É essa lacuna que a observabilidade fecha — e é com ela que se conclui a esteira DevOps completa.

1.4 Como organizar o seu tempo: em aula e em casa

Este é o laboratório mais longo da disciplina. Por isso, ele foi dividido em uma parte feita **em aula** e uma parte feita **em casa**, até a aula de fechamento (19/10). Cada seção e cada exercício traz a marcação *(em aula)* ou *(em casa)*.

Onde	O que fazer	Por quê
Em aula	Seção 3, Etapas 1 a 4 (rede, cAdvisor, Prometheus e Grafana) · Seção 4 (fonte de dados e painel de CPU) · Seção 5 (incidente) · Exercícios 1, 2 e 3	É o ciclo funcional: você vê a métrica nascer no container e o pico aparecer no gráfico.
Em casa	Seção 3, Etapa 5 (enviar ao GitHub) · Seção 6 (mini-postmortem) · Exercícios 4 a 7, incluindo o alerta do Grafana (sugestão: no feriado de 12/10)	São tarefas de escrita e de exploração, que rendem mais sem a pressão da aula. Todas entram no seu relatório de conclusão.

Baixe as imagens novas deste laboratório. Isso economiza muito tempo em sala e evita o erro `toomanyrequests`, que o Docker Hub devolve quando muitos computadores baixam imagens ao mesmo tempo pela mesma rede. Na linha de comando, um comando por vez:

```
docker pull gcr.io/cadvisor/cadvisor:latest

docker pull prom/prometheus:latest

docker pull grafana/grafana:latest

docker pull busybox
```

As imagens `nginx` e `postgres:16` você já baixou no Lab 1.

2. O que é observabilidade

Observabilidade é a capacidade de entender **o que está acontecendo dentro** de um sistema apenas observando seus sinais externos, apoiada em três pilares:

Pilar	Pergunta que responde	Exemplo de ferramenta
Métricas	"O quanto?" (CPU, memória, requisições/seg)	Prometheus

Pilar	Pergunta que responde	Exemplo de ferramenta
Logs	"O que aconteceu, em detalhe?"	ELK Stack, Loki
Traces	"Por onde a requisição passou e onde demorou?"	Jaeger, Zipkin

> **Por que este laboratório foca só em métricas, e não nos três pilares?** Métricas são o pilar mais barato de instrumentar em cima de uma infraestrutura já existente — o cAdvisor lê métricas de containers **sem precisar de nenhuma mudança no código da aplicação**, diferente de logs estruturados ou traces, que normalmente exigem instrumentar a aplicação em si. Por isso, métricas costumam ser o primeiro pilar de observabilidade que uma equipe implementa.

Nesta apostila focamos em **métricas**, usando **Prometheus** (coleta e armazena as métricas) e **Grafana** (visualiza em dashboards).

3. Adicionando a stack de observabilidade ao `main.tf`

> **Antes de começar:** confirme que o ambiente dos Labs 1 e 2 está de pé. No terminal, `docker ps` deve listar `meu-container-web` e `meu-container-db`, e `curl http://localhost:8080` deve mostrar o **seu** HTML (não a página padrão de boas-vindas do Nginx). Se algo falhar aqui, resolva antes — este laboratório constrói em cima disso.

> **Por que uma rede Docker dedicada?** Até aqui, os containers do Lab 1 conversavam entre si pela rede padrão do Docker. Ao criar uma rede própria (`monitoring-net`) e conectar a ela os serviços de observabilidade, garantimos que Prometheus, Grafana e cAdvisor consigam se encontrar **pelo nome do container**, de forma isolada e previsível — o mesmo princípio de organização que, em um ambiente real, separaria a rede de monitoramento da rede de aplicação. Atenção a um detalhe: a rede serve para os serviços *se encontrarem*. Quem lê o consumo dos containers é o cAdvisor, e ele enxerga **todos** os containers do computador, estejam ou não nessa rede.

Vamos montar a stack em **quatro etapas** (mais uma quinta, para enviar tudo ao GitHub), aplicando e conferindo o resultado a cada uma. Assim, se algo der errado, você sabe exatamente em qual etapa foi o erro. Em todas elas, abra o `main.tf` no VS Code, faça a alteração, salve com `Ctrl+S` e só então rode os comandos no terminal integrado.

Etapa 1 — Rede e container `web` *(em aula)*

Acrescente o bloco `docker_network` ao `main.tf` e **substitua** o bloco `resource "docker_container" "web"` que você já tem pela versão abaixo. Ela é a mesma do Lab 1 com uma única adição, o `networks_advanced`. **Mantenha o bloco `volumes`** — se ele for apagado, o Nginx volta a mostrar a página padrão em vez do seu `site/index.html`.

```
resource "docker_network" "monitoring" {
  name = "monitoring-net"
}

resource "docker_container" "web" {
  name  = "meu-container-web"
  image = docker_image.nginx.image_id
  ports {
```

```
    internal = 80
    external = 8080
  }
  volumes {
    host_path      = "${path.cwd}/site"
    container_path = "/usr/share/nginx/html"
    read_only      = true
  }
  networks_advanced {
    name = docker_network.monitoring.name
  }
}
```

No terminal integrado, digite — um comando por vez:

```
terraform apply
```

Digite `yes` quando o Terraform pedir confirmação. Ele vai avisar que o container `web` "must be replaced" (será recriado) — isso é esperado, porque o container mudou de rede. Para conferir:

```
docker ps

curl http://localhost:8080
```

O `curl` deve mostrar o seu HTML. Se mostrar a página padrão do Nginx, o bloco `volumes` ficou de fora — volte ao `main.tf` e confira.

Etapa 2 — cAdvisor *(em aula)*

> **Por que o cAdvisor?** Ele lê diretamente do Docker (uso de CPU, memória, rede e disco de cada container) sem exigir nenhuma configuração adicional nos containers do Lab 1 — é o jeito mais rápido de ter métricas de infraestrutura sem tocar em nada que já existe. Para isso, ele precisa enxergar o "lado de dentro" do computador (as pastas `/sys`, `/var/lib/docker` e outras): é para isso que servem os vários blocos `volumes` e o `privileged` abaixo. **Sem eles, o cAdvisor sobe sem erro, mas não identifica os containers pelo nome — e o painel do Grafana fica vazio.**

Acrescente o bloco abaixo ao `main.tf` e salve:

```
resource "docker_container" "cadvisor" {
  name       = "cadvisor"
  image      = "gcr.io/cadvisor/cadvisor:latest"
  privileged = true
  ports {
    internal = 8080
    external = 8081
  }
```

```
  volumes {
    host_path      = "/"
    container_path = "/rootfs"
    read_only      = true
  }
  volumes {
    host_path      = "/var/run"
    container_path = "/var/run"
    read_only      = true
  }
  volumes {
    host_path      = "/sys"
    container_path = "/sys"
    read_only      = true
  }
  volumes {
    host_path      = "/var/lib/docker"
    container_path = "/var/lib/docker"
    read_only      = true
  }
  volumes {
    host_path      = "/dev/disk"
    container_path = "/dev/disk"
    read_only      = true
  }
  devices {
    host_path      = "/dev/kmsg"
    container_path = "/dev/kmsg"
    permissions    = "rwm"
  }
  networks_advanced {
    name = docker_network.monitoring.name
  }
}
```

No terminal, digite `terraform apply` (e `yes`). Depois, **confira no navegador**: abra `http://localhost:8081/containers/`. Na seção dos containers Docker devem aparecer `meu-container-web` e `meu-container-db`. Se a página abrir mas não listar nenhum container, os blocos `volumes` não foram copiados corretamente — revise o bloco acima antes de seguir.

Etapa 3 — Prometheus *(em aula)*

> **Por que o Prometheus, e não o Grafana lendo o cAdvisor diretamente?** O cAdvisor expõe métricas apenas do **instante atual** — ele não guarda histórico. O Prometheus é quem periodicamente "puxa" (*scrape*) essas métricas e as armazena ao longo do tempo, o que é o que torna possível ver um gráfico de CPU "nos últimos 10 minutos", e não só o valor de agora.

No VS Code, no Explorer, clique com o botão direito na **raiz** do projeto (a pasta `meu-projeto`, ao lado do `main.tf`) → **New File** → nomeie `prometheus.yml`. Cole o conteúdo abaixo e salve com `Ctrl+S`:

`prometheus.yml`:

```
global:
  scrape_interval: 5s

scrape_configs:
  - job_name: 'cadvisor'
    static_configs:
      - targets: ['cadvisor:8080']
```

Confira no terminal que ele foi criado como **arquivo**, na pasta certa:

```
ls -l prometheus.yml
```

A linha impressa deve começar com `-` (arquivo). Se começar com `d`, o Docker criou uma **pasta** com esse nome — apague-a (`rm -r prometheus.yml`), crie o arquivo de novo e repita. Se aparecer "No such file", você está na pasta errada: use `cd ~/meu-projeto`.

De volta ao `main.tf`, **acrescente** os dois blocos abaixo e salve:

```
resource "docker_image" "prometheus" {
  name = "prom/prometheus:latest"
}

resource "docker_container" "prometheus" {
  name  = "prometheus"
  image = docker_image.prometheus.image_id
  ports {
    internal = 9090
    external = 9090
  }
  volumes {
    host_path      = "${path.cwd}/prometheus.yml"
    container_path = "/etc/prometheus/prometheus.yml"
  }
  networks_advanced {
    name = docker_network.monitoring.name
  }
}
```

Digite `terraform apply` (e `yes`). Para conferir, abra `http://localhost:9090/targets` no navegador: o alvo `cadvisor` deve aparecer com o estado **UP** (pode levar uns 10 segundos; atualize a página). Se aparecer **DOWN**, confira o nome e a porta em `targets: ['cadvisor:8080']`.

> **Atenção:** se você editar o `prometheus.yml` depois de o container estar no ar, a mudança **não** é percebida sozinha. Recrie o container com `terraform apply -replace=docker_container.prometheus`.

Etapa 4 — Grafana *(em aula)*

> **Por que o Grafana, se o Prometheus já guarda o histórico?** O Prometheus foi feito para armazenar e consultar séries temporais de forma eficiente, não para exibi-las de forma amigável. O Grafana é a camada de **visualização**: painéis, gráficos e alertas configuráveis por cima dos dados que o Prometheus já coleta — reaproveitando, note bem, o mesmo padrão de "cada ferramenta faz uma coisa bem feita" que você já viu na relação Docker/Terraform no Lab 1.

Acrescente o último bloco da stack ao `main.tf` e salve:

```
resource "docker_image" "grafana" {
  name = "grafana/grafana:latest"
}

resource "docker_container" "grafana" {
  name  = "grafana"
  image = docker_image.grafana.image_id
  ports {
    internal = 3000
    external = 3000
  }
  networks_advanced {
    name = docker_network.monitoring.name
  }
}
```

Digite `terraform apply` (e `yes`) e depois `docker ps`: devem estar no ar `meu-container-web`, `meu-container-db`, `cadvisor`, `prometheus` e `grafana`. Abra `http://localhost:3000` para conferir que a tela de login do Grafana aparece.

Etapa 5 — Enviando para o GitHub (a pipeline do Lab 2) *(em casa)*

Como o repositório agora tem a pipeline do Lab 2, as novidades precisam ir para o GitHub — inclusive o `prometheus.yml`. No terminal integrado, um comando por vez:

```
terraform fmt

git add main.tf prometheus.yml

git commit -m "Adiciona stack de observabilidade"

git push
```

O `terraform fmt` arruma a formatação do `main.tf` — a pipeline roda `terraform fmt -check` e falha se encontrar algo fora do padrão (lembra do "falhar cedo" do Lab 2?). Se o `fmt` listar algum arquivo, ele foi corrigido: salve e inclua no `git add`. **Não esqueça o `prometheus.yml`**: sem ele no repositório, a pipeline ficaria verde, mas o Prometheus não subiria no runner. Acompanhe a execução na aba **Actions**.

> Lembre-se: o runner do GitHub é temporário. Os serviços que a pipeline sobe lá **não** podem ser abertos no seu navegador — a stack que você usa nas próximas seções é a da sua máquina.

4. Configurando o dashboard *(em aula)*

1. Acesse `http://localhost:3000` (usuário/senha padrão: `admin` / `admin`). No primeiro acesso, o Grafana pede para você criar uma nova senha — pode clicar em **Skip** para pular.

2. **Conecte o Prometheus:** no menu lateral, **Connections > Data sources > Add data source > Prometheus**. No campo **URL**, digite `http://prometheus:9090` (o nome do container na rede Docker, **não** `localhost`). Role até o fim da página e clique em **Save & test**: deve aparecer uma mensagem verde de sucesso.

3. **Monte um painel de CPU:** no menu lateral, **Dashboards > New > New dashboard > Add visualization**. Escolha o data source **Prometheus**. Na aba **Queries**, alterne para o modo **Code** e cole a consulta abaixo (ela mostra a CPU do Nginx e do container de teste que usaremos na seção 5), depois clique em **Run queries**:

   ```
   rate(container_cpu_usage_seconds_total{name=~"meu-container-web|estresse"}
   [1m])
   ```

4. Dê um título ao painel (por exemplo, "CPU dos containers"), clique em **Apply** e depois em **Save dashboard**. No canto superior direito do dashboard, ajuste o intervalo de tempo para **Last 15 minutes** e o *refresh* para **5s** — sem isso, um pico de 1 minuto quase não aparece no gráfico.

> **Como ler o gráfico:** o valor é medido em *núcleos de CPU*: `1` significa um núcleo inteiro ocupado, `0.5` meio núcleo. Com o Nginx parado, a linha de `meu-container-web` fica perto de zero.

> **Por que `rate(...)` e não só o nome da métrica?** `container_cpu_usage_seconds_total` é um **contador acumulado**: guarda o total de segundos de CPU gastos desde que o container nasceu, e por isso só sobe — se você plotar o valor cru, a linha nunca "volta ao normal". A função `rate(...[1m])` calcula quanto desse total foi gasto *por segundo* na última janela de 1 minuto, que é o que representa o uso de CPU naquele momento.

> **Opcional:** em vez de montar o painel, você pode importar um dashboard pronto de cAdvisor (**Dashboards > New > Import**, ID `14282`, **Load**). Na tela seguinte, escolha **Prometheus** na lista de data source antes de clicar em **Import** — se esquecer, o painel mostra "No data". Se mesmo assim ficar vazio, volte à conferência do cAdvisor na seção 3 (Etapa 2).

> **Por que `http://prometheus:9090` e não `http://localhost:9090`?** Dentro da rede Docker `monitoring-net`, cada container tem seu nome resolvido automaticamente como se fosse um hostname — `localhost`, de dentro do container do Grafana, apontaria para o próprio Grafana, não para o Prometheus.

5. Simulando um incidente *(em aula)*

Deixe o painel de CPU aberto no navegador (intervalo de 15 minutos, *refresh* de 5 s). Em um terminal, gere carga artificial digitando:

```
docker run --rm --init --name estresse --network monitoring-net busybox timeout 60
md5sum /dev/zero
```

Na primeira vez, o Docker baixa a imagem `busybox` (bem pequena) — isso leva alguns instantes antes de a carga começar. Durante os 60 segundos seguintes o terminal fica parado e sem mensagens: **é assim mesmo**, olhe para o navegador. Em até alguns segundos a linha do container `estresse` sobe para perto de `1` (um núcleo inteiro ocupado). Quando os 60 segundos terminam, o prompt volta sozinho, o container é removido e a linha do `estresse` deixa de ser desenhada (o container deixou de existir), enquanto a do `meu-container-web` continua perto de zero.

> **O que esse comando faz de verdade?** Ele cria um **container novo**, chamado `estresse`, que fica "queimando" um núcleo de CPU por 60 segundos. Ele não mexe no container `web`; serve como uma carga de trabalho que sai do controle dentro do ambiente — por isso o nome `estresse` está na consulta do painel. Peça por peça: `busybox` é uma imagem oficial minúscula com utilitários básicos; `md5sum /dev/zero` calcula o resumo (*hash*) de um fluxo infinito de zeros, o que mantém um núcleo de CPU ocupado sem parar; `timeout 60` interrompe esse cálculo depois de 60 segundos; `--rm` apaga o container ao final; `--init` é necessário para o `timeout` conseguir encerrar o processo (sem ele, o container não parava sozinho). O teste ocupa um único núcleo: como a VM tem 2 CPUs, sobra um para o Grafana e o navegador continuarem respondendo.

6. Da detecção à resposta: um mini-postmortem *(em casa)*

Após simular o incidente, documente:

- **O que aconteceu:** descrição objetiva do sintoma observado.
- **Quando foi detectado:** horário em que o dashboard mostrou o desvio.
- **Causa raiz:** o que gerou o pico (no caso, o comando `stress`).
- **Ação tomada:** o que foi feito para mitigar.
- **Como evitar no futuro:** que alerta ou automação preveniria/aceleraria a resposta a um caso real.

Esse formato — usado por praticamente toda equipe DevOps em produção — é chamado de **postmortem sem culpa** (*blameless postmortem*): o foco é o processo e o sistema, nunca em apontar culpados.

7. Exercícios de prática — Lab 3

Os exercícios 1, 2 e 3 são feitos **em aula**; os exercícios 4 a 7 são feitos **em casa**, até 19/10.

1. **Subindo a stack:** execute o `terraform apply` com os novos recursos e confirme, via `docker ps`, que `cadvisor`, `prometheus` e `grafana` estão no ar. Confirme também que `http://localhost:9090/targets` mostra o alvo `cadvisor` como **UP**.

2. **Conectando a fonte de dados:** configure o Prometheus como *data source* no Grafana e explique por que a URL usada é `http://prometheus:9090` e não `http://localhost:9090`.

3. **Montando um painel:** crie um painel no Grafana mostrando o uso de CPU do container `meu-container-web` ao longo do tempo, usando a consulta da seção 4. Explique por que ela usa `rate(...)` em vez da métrica pura.

4. **Criando um alerta (tarefa assíncrona — entregar até a aula de fechamento):** configure uma regra de alerta no Grafana para disparar quando o uso de CPU do container ultrapassar um limite definido por você. Explique sua escolha de limiar. **Dica:** a consulta do alerta precisa incluir o container `estresse` (o que recebe a carga na seção 5), e o valor da CPU é medido em núcleos (`1` = um núcleo inteiro) — escolha o limiar nessa unidade. Teste o alerta rodando o comando da seção 5. Como envolve explorar com calma uma interface nova, este exercício não é feito durante a aula: faça em casa e traga o print da regra configurada, mais a justificativa do limiar escolhido, como parte do entregável final.

5. **Incidente e postmortem:** rode o comando de `stress` da seção 5 e escreva um mini-postmortem seguindo o roteiro da seção 6.

6. **Métrica de detecção (MTTD):** meça, com um cronômetro, quanto tempo levou entre o início do incidente simulado e o momento em que você "percebeu" o pico no dashboard. Proponha uma melhoria (ex.: alerta automático por e-mail/Slack) que reduziria esse tempo em um cenário real.

7. **Observando o banco de dados:** o container `meu-container-db` (do Lab 1) **não** foi conectado à rede `monitoring-net`. Antes de alterar qualquer coisa, anote sua hipótese: o uso de CPU/memória dele aparece no cAdvisor? Teste trocando o filtro da consulta do painel para incluir `meu-container-db` (ou abrindo `http://localhost:8081/containers/`). Depois responda: a rede `monitoring-net` é necessária para o cAdvisor enxergar um container? Para quem ela é necessária, de fato? Em seguida, acrescente ao bloco `docker_container "db"` do `main.tf` o trecho abaixo, rode `terraform apply` e observe o que muda (e o que *não* muda) — inclusive se a mensagem que você inseriu no banco no Lab 1 continua lá, e por quê:

   ```
   networks_advanced {
     name = docker_network.monitoring.name
   }
   ```

   O que isso sugere sobre a abrangência da observabilidade baseada em containers, independentemente de o serviço ser stateless ou stateful?

8. Encerramento

Com este laboratório, você fecha os três grandes desafios que compõem a esteira DevOps: **provisionar** infraestrutura de forma declarativa e repetível (Lab 1), **automatizar** sua aplicação a cada mudança (Lab 2) e **observar e reagir** quando algo sai do esperado (Lab 3). Ao concluir os três laboratórios, você deve ser capaz de:

- Explicar o que é DevOps, os pilares CALMS e por que essa cultura surgiu;
- Provisionar infraestrutura de forma declarativa com Terraform e Docker, incluindo serviços stateless e stateful (Lab 1);
- Automatizar a aplicação dessa infraestrutura em uma pipeline de CI/CD com GitHub Actions (Lab 2);
- Instrumentar essa infraestrutura com métricas e reagir a um incidente com apoio de Prometheus e Grafana (Lab 3);
- Reconhecer, na prática, conceitos como *drift de configuração*, *idempotência*, *fail fast* e *postmortem sem culpa*.

> **Para ir além:** pesquise como esses mesmos três desafios aparecem em provedores de nuvem reais (AWS, Azure, GCP) — o provedor do Terraform muda, a pipeline de CI/CD ganha etapas de segurança adicionais, e a stack de observabilidade pode virar um serviço gerenciado (ex.: CloudWatch, Azure Monitor) — mas a esteira **IaC → CI/CD → Observabilidade** permanece exatamente a mesma.

9. Entregável final (aula de fechamento — vale nota da P1)

Na aula de fechamento, você entrega um único pacote consolidando os três laboratórios. Ele deve conter:

1. O link do repositório GitHub (do Lab 2), com o `main.tf` **completo** — os recursos do Lab 1 (Nginx, Postgres) e do Lab 3 (rede, cAdvisor, Prometheus, Grafana) já integrados — mais `site/index.html`, `prometheus.yml` e `.github/workflows/terraform.yml`.
2. Um print de pelo menos uma execução **verde** na aba Actions do GitHub (Lab 2).
3. O mini-postmortem da seção 6 deste laboratório.
4. O print e a justificativa do alerta do Grafana (exercício 4 desta lista, feito de forma assíncrona).
5. As respostas aos exercícios dos três laboratórios que o professor indicar.

Chegar a essa entrega é o que fecha, formalmente, a esteira **provisionar → automatizar → observar** percorrida nos três laboratórios — e é isso que será avaliado como nota da P1.